

Vivekanand Education Society's Institute Of Technology (VESIT)

Department of Information Technology

Course: **Blockchain**

Mini Project

Cookbook / Operations Manual

MeshSearch

*Privacy-Preserving Decentralized Web Intelligence Retrieval
Using Zero-Knowledge Query Commitments, Multi-Hop Onion Routing,
and Blockchain-Mediated Micropayment Settlement*

Roll No.	Name
16	Jay Kerkar
17	Yash Kolekar

Group No.: 5

Date: September 8, 2026

Faculty In-Charge: Kajal Jewani

1 Overview

MeshSearch is a privacy-preserving web search system exposed to AI agents as an MCP (Model Context Protocol) server. Queries are committed client-side with zero-knowledge (ZK) proofs before leaving the device, payment is settled anonymously per search using the x402 micropayment protocol, traffic is routed through a 3-hop ENS-named onion relay network, relay operators are paid through unlinkable BitGo stealth addresses, and search history is stored encrypted on Fileverse (IPFS-backed).

1.1 Privacy Layers

1. **Client** – Poseidon hash commitment of the query; raw text never leaves the device.
2. **MCP Server** – verifies ZK proof and x402 payment; never sees plaintext.
3. **Relay Network** – 3-hop onion routing (`relay1/2/3.meshsearch.eth`); each hop knows only prev/next.
4. **Search Backend** – self-hosted SearXNG; the only node that decrypts the query.
5. **Storage** – Fileverse, encrypted with the user's wallet key.

1.2 Core Stack

Layer	Tooling
ZK	Semaphore v4 (Groth16, Poseidon)
Crypto	@noble/curves secp256k1 ECDH, @noble/hashes HKDF + AES-256-GCM
Payments	x402 (@x402/core, @x402/evm), USDC on Base Sepolia
Wallets	BitGo SDK, self-custody MPC hot wallets, Hoodi testnet
Blockchain	Solidity, Hardhat, ethers v6
MCP	@modelcontextprotocol/sdk (Streamable HTTP + stdio)
Search	SearXNG (self-hosted, Dockerized)
Monorepo	pnpm workspaces, Turborepo

2 Prerequisites

- Node.js ≥ 20
- pnpm ≥ 9
- Docker (required for the SearXNG container)
- A Base Sepolia wallet with test ETH and test USDC (for payment flows)

2.1 Install Dependencies

```
git clone https://github.com/WhyAsh5114/MeshSearch.git
cd MeshSearch
pnpm install
```

3 Setup

3.1 One-Command Setup (recommended)

```
pnpm run:setup
```

This interactive script performs the full bootstrap:

1. Checks prerequisites (**node**, **pnpm**, **docker**).
2. Generates secp256k1 key pairs for all services.
3. Configures ports (SearXNG 8888, relays 4002-4004, MCP 3038).
4. Deploys smart contracts to Base Sepolia (local Hardhat optional).
5. Optionally enables x402 payments.
6. Writes all **.env** files.
7. Starts SearXNG, the 3 relay nodes, the search backend, Fileverse, and the MCP server.
8. Writes the MCP wiring config to **mcp-config.json**.

3.2 Manual Contract Deployment (Base Sepolia)

```
cd packages/contracts
export BASE_SEPOLIA_RPC_URL=https://sepolia.base.org
export DEPLOYER_PRIVATE_KEY=0x<your-private-key>
pnpm deploy
```

The script prints BaseScan links for every deployed contract (NodeRegistry, NullifierRegistry, PaymentSplitter, AccessControl).

3.3 Local-Only Fallback

If RPC/contract configuration is unavailable, **private_search** fails fast by default. For local testing only:

```
export MESHSEARCH_ALLOW_INMEMORY_FALLBACK=true
```

4 Configuration

4.1 x402 Micropayments

Set in **apps/mcp-server/.env**:

```
X402_ENABLED=true
X402_PAY_TO=0xYourWalletAddress
X402_SEARCH_PRICE=$0.001
X402_NETWORK=eip155:84532
```

When enabled, **private_search** returns HTTP 402 with a USDC price quote; the client signs a USDC authorization and re-sends the request, and the server settles on-chain.

4.2 BitGo Stealth Disbursement (optional)

```
BITGO_ACCESS_TOKEN=<testnet-token>
BITGO_WALLET_ID=<treasury-wallet-id>
BITGO_WALLET_PASSPHRASE=<wallet-passphrase>
BITGO_ENV=test
```

```
BITGO_COIN=hteth
BITGO_EXPRESS_URL=http://localhost:3080
RELAY1_BITGO_WALLET_ID=<relay1-wallet-id>
RELAY2_BITGO_WALLET_ID=<relay2-wallet-id>
RELAY3_BITGO_WALLET_ID=<relay3-wallet-id>
```

Bootstrap steps:

1. Create a test account at `app.bitgo-test.com`.
2. Run BitGo Express locally: `docker run -p 3080:3080 bitgo/express:latest`.
3. Create wallets and fund them from a Hoodi faucet.

When BitGo is not configured, disbursement falls back to the on-chain `PaymentSplitter` contract.

5 Running the System

5.1 Start Every Service Together

```
pnpm dev
```

Starts the relay nodes, search backend, MCP server, and web app in one process, from the monorepo root.

5.2 Start the Web App Only

(backend services must already be running – see Section 3)

```
cd apps/web && pnpm dev
# open http://localhost:3000
```

5.3 Connect to Claude Desktop / Cursor

Copy the generated `mcp-config.json` into:

- **Claude Desktop (macOS):**
`~/Library/Application Support/Claude/claude_desktop_config.json`
- **Claude Desktop (Linux):**
`~/.config/claude/claude_desktop_config.json`
- **Cursor:** Settings → MCP Servers

Alternatively, connect over HTTP transport directly at `http://localhost:3038/mcp`.

6 MCP Tools Reference

Tool	Description
<code>private_search</code>	ZK-committed search with onion routing and x402 payment gate.
<code>get_history</code>	Retrieves and decrypts the caller's search history from Fileverse.
<code>compile_report</code>	Aggregates prior searches into an encrypted Fileverse document.

Request lifecycle for `POST /mcp`: the body is buffered, the JSON-RPC method is peeked, and only `tools/call` passes through the x402 gate – `initialize`, `tools/list`, and notifications bypass payment.

7 Demo Client (Interactive x402 REPL)

```
cd apps/mcp-server
export TEST_WALLET_KEY=0x<your-private-key>

# Interactive mode
pnpm demo

# Single query
node demo.mjs "what are zero knowledge proofs"
```

REPL commands: type a query to search, **balance** to check USDC, **help** for options. The client loads a wallet (viem), opens an MCP session, sends an unpaid **tools/call** to receive a 402 price quote, signs the USDC authorization, re-sends with a **PAYMENT-SIGNATURE** header, and prints the BaseScan settlement link.

Testnet faucets:

- ETH (gas): alchemy.com/faucets/base-sepolia
- USDC: faucet.circle.com

8 Testing

```
pnpm turbo test
```

Runs the full suite (95+ tests) across crypto primitives, contracts, middleware, relay routing, storage, and x402 payment flows. An automated end-to-end payment check is also available:

```
node apps/mcp-server/test-x402.mjs
```

9 Viewing & Monitoring

9.1 Health Check

```
curl http://localhost:3038/health
```

Returns server status, active session count, and a bitgo block (**status**, **env**, **coin**, **walletId**, **expressUrl**, **relayWallets**).

9.2 MCP Inspector

Navigate to <http://localhost:3000/mcp> for a live debug view of MCP requests, latency stats, full request/response payloads, and the BitGo status badge.

9.3 Dashboard (optional)

```
cd apps/dashboard && pnpm dev
# open http://localhost:3001
```

Shows the relay map, payment visualization, and history.

10 Smart Contracts

Four Solidity contracts on Base L2, tested with Hardhat (`packages/contracts/contracts/`):

Contract	Purpose	Key Functions
NodeRegistry	Relay registration, ENS linking, reputation	<code>registerNode()</code> , <code>updateReputation()</code>
NullifierRegistry	Stores used nullifiers, prevents replay	<code>recordNullifier()</code> , <code>isNullifierUsed()</code>
PaymentSplitter	Distributes x402 payments to relays	<code>split()</code>
AccessControl	ENS subscription tier management	<code>checkAccess()</code> , <code>grantTier()</code>

```
cd packages/contracts
pnpm hardhat test # contract test suite
pnpm hardhat compile # compile contracts
```

11 Privacy Threat Model

Threat	Mitigation
Search engine logs the query	SearXNG is self-hosted with zero logs; query arrives pre-encrypted
MCP server sees the query	Server only sees the Poseidon commitment hash, never plaintext
Payment linked to query	Semaphore nullifiers unlink the wallet from search events
Network observer sees traffic	3-hop onion routing; each relay knows only prev/next hop
Storage provider reads history	Fileverse entries are encrypted with the user's wallet key
Relay operator tampers	On-chain reputation tracking; bad actors lose score
Replay attack	Nullifiers stored on-chain; reused nullifiers are rejected
Result manipulation	Result hash stored on-chain for tamper-evident verification
Payment links relay identity	BitGo fresh stealth address per disbursement

12 Services & Ports

Service	Port	Package	Role
SearXNG	8888	Docker	Self-hosted metasearch engine
Search Backend	4001	apps/search-backend	SearXNG wrapper, result hashing
Relay 1	4002	apps/relay-node	Onion routing hop
Relay 2	4003	apps/relay-node	Onion routing hop
Relay 3	4004	apps/relay-node	Onion routing hop
Fileverse	4005	apps/fileverse	Encrypted document storage
MCP Server	3038	apps/mcp-server	Main entry point (x402 + MCP + BitGo)
BitGo Express	3080	Docker	Local MPC key management
Web App	3000	apps/web	Chat UI, x402, BitGo status
Dashboard	3001	apps/dashboard	Relay map, network stats

13 Troubleshooting

Symptom	Fix
private_search fails immediately	RPC/contract config missing – set <code>BASE_SEPOLIA_RPC_URL</code> or enable <code>MESHSEARCH_ALLOW_INMEMORY_FALLBACK</code> for local testing.
402 loops forever	Wallet has no test USDC – fund from <code>faucet.circle.com</code> .
BitGo status shows disabled	Confirm <code>BITGO_ACCESS_TOKEN</code> and <code>BITGO_EXPRESS_URL</code> are set and BitGo Express is running on port 3080.
SearXNG unreachable	Confirm the Docker container is running and port 8888 is free.

14 Repository Map

```
apps/
  mcp-server/ MCP server (x402 gate, ZK verification, BitGo wallets)
  relay-node/ 3-hop onion routing relays (ENS-named)
  search-backend/ SearXNG wrapper + result hashing
  fileverse/ Encrypted history storage (IPFS)
  web/ Next.js browser UI
  dashboard/ Relay map / network stats (optional)
packages/
  contracts/ NodeRegistry, NullifierRegistry, PaymentSplitter, AccessControl
  crypto/ secp256k1 ECDH, AES-256-GCM, Semaphore ZK proofs
  types/ Shared TypeScript types
scripts/
  setup.ts Interactive setup & launcher
```

Source: <https://github.com/WhyAsh5114/MeshSearch>